

Commons Architecture - Deep Dive

Scope: In this document, the Commons means the TypeScript Commons Core implementation itself. The code is treated as the system being analyzed, not merely as a database beneath a separate Commons product.

1. Core architectural identity

The TypeScript establishes the Commons as a software system whose central coordinator is CommonsCore. It combines immutable tables, participants, context, history, persistence, and an LM Studio interface. No single component is the Commons by itself; CommonsCore coordinates the components into one operating system.

2. CommonsCore as the kernel

CommonsCore is the system boundary and primary coordinator. It owns or manages tables, participants, version history, contextual state, persistence, and the command surface. This makes CommonsCore the closest equivalent to a kernel: it defines which subsystems exist, how they interact, and where authority is centralized.

3. ImmutableTable as the authoritative record subsystem

Each RecordEntry contains a record_id, data, and provenance. Provenance includes created_by, timestamp, and table_id. Insert operations create new records instead of modifying existing ones. The result is an append-only record structure that supports traceability and auditability.

4. Provenance and audit identity

Provenance is not decorative metadata. It is the link between a record and the participant, time, and table responsible for its creation. That gives the Commons a foundation for answering who added something, when it was added, and where it belongs. A mature version would likely add source references, operation IDs, and links to the context or rule that authorized the operation.

5. VersionHistoryEngine

VersionHistoryEngine records snapshots of Commons state using a version UUID, timestamp, author, and state snapshot. Record provenance answers questions about individual records; version history answers what the Commons looked like at a particular point. The current CLI does not automatically commit every state-changing operation, so this mechanism is present but not fully integrated.

6. ContextLayer

ContextLayer creates named contexts with scope, observations, and interpretations. It gives the Commons a place to organize the working conditions around a task without confusing those conditions with the permanent record store. In the current implementation, contexts are in memory only and disappear when the process ends.

7. LocalToolkit

LocalToolkit is an operational utility boundary. Its JSON Patch support handles simple add operations, and its schema validation checks for required keys. These functions are useful, but they are not governance. A toolkit operation describes how something can be done; Commons governance determines whether a participant is allowed to do it.

8. LMStudioClient as the AI boundary

LMStudioClient is a thin adapter to the local LM Studio OpenAI-compatible API. The default target is <http://localhost:1234/v1> and the client calls `/chat/completions` with model, messages, and temperature. This separation is architecturally important: the model contributes intelligence, but the model conversation is not the authoritative state of the Commons.

9. Participants and authority

The participant registry currently stores a name and a HUMAN or AI type. That creates an identity abstraction, but not yet a complete authorization model. The architectural direction is clear: an AI may propose, interpret, or transform information without automatically receiving unrestricted authority to alter authoritative Commons state.

10. Persistence and source of truth

The current implementation serializes tables, participants, and history into `commons_state.json` using synchronous file writes. That is adequate for a prototype but weak for concurrent access, crash safety, large state, and partial writes. More importantly, the persistent Commons state should remain the source of truth rather than relying on an active LM Studio conversation to preserve knowledge.

11. The Leo transfer problem

If Leo depends on a long LM Studio conversation to hold the effective knowledge state, breaking that conversation can destroy the practical working context. The Commons architecture should reverse that dependency: persistent Commons state should be loaded into a new working context whenever a session begins. The model session becomes temporary; the Commons state remains.

12. Memory versus context

Persistent memory and working context should remain distinct. Persistent memory can store identity, configuration, durable knowledge, and tools. Working context can contain the current task, selected records, observations, relevant interpretations, active rules, and recent history. A session is therefore a projection assembled from persistent sources, not the permanent storage location itself.

13. Raw records, interpretations, and knowledge

The architecture naturally supports a distinction between raw records, provenance, contextual observations, interpretations, derived knowledge, and eventually established knowledge. An AI-generated interpretation should not silently become an authoritative fact merely because a model generated it. Authority, status, and confidence are separate dimensions and should remain separate in indexing and state representation.

14. Novelty and knowledge lifecycle

The broader Commons design distinguishes novelty, importance, and relevance. The planned novelty lifecycle is New -> Emerging -> Established -> Dormant -> Expired. This gives the system a way to keep persistent knowledge while controlling what remains active, without treating every item as equally important forever.

15. Architectural layering

A mature TypeScript Commons can be understood as a set of cooperating layers: governance, identity and participants, context, immutable records, history, persistent storage, and the AI interface. The current prototype already contains pieces of these layers. The remaining work is to make their boundaries explicit and enforce their responsibilities consistently.

16. What the prototype already does well

The strongest current elements are immutable records, basic provenance, centralized coordination through CommonsCore, an explicit version-history mechanism, an explicit context abstraction, persistent state serialization, and a decoupled LM Studio client. These are meaningful architectural foundations rather than isolated utilities.

17. Current gaps

The main gaps are durable context persistence, schema enforcement at insertion, real authentication and authorization, automatic version commits, transaction-safe storage, richer provenance, context retrieval, complete memory integration, and the larger governance machinery described by the Commons concept. These are implementation gaps inside the Commons, not evidence that the architecture needs to be replaced.

18. Central principle

The central principle is that the AI does not become the state. The conversation does not become the state. The model context window does not become the state. The Commons is the persistent, governed system of record, while an AI receives a working view of that state and contributes intelligence back through controlled operations.

19. Bottom line

The TypeScript is the Commons in this implementation. CommonsCore is its kernel. ImmutableTable provides the authoritative append-only record mechanism; provenance establishes traceability; VersionHistoryEngine provides state history; ContextLayer provides contextual organization; LocalToolkit provides operations; persistence preserves state; and LMStudioClient provides the AI boundary. The next architectural step is to strengthen this code so that its state, context, history, authority, and recovery mechanisms become durable and session-independent.